EOS 4.71-beta2

Operation Manual Addendum

This is the Operation Manual Addendum for **EOS version 4.71-beta2**, an unofficial update of E-mu's EOS 4.70 for the **E4 Ultra** family of synthesizer/samplers. It includes everything in EOS 4.71-beta: Oberheim OB-Xa filter models, tempo-synced LFOs, transwave scanning of sample loops, a MIDI clock that keeps running when your sequencer stops, and a fix for the pan controls.

New in 4.71-beta2: the **4 Pole LP+Fuzz** filter and smoother resonance steps, **Insert Silence** in Sample Edit, the **'Slice** cord destination, a **Slice** key in Beat Munger that turns a loop into a sliced preset, **WTScan** improvements (notes start at the loop, fast moves glide), and a balanced pan centre.

New in +LfoCk: Lfo1Ck and **Lfo2Ck** cord destinations. A negative amount selects that LFO's tempo division (4 bars through 1/32) and overrides Rate. While it does, the Rate field shows that division and follows the controller, and the knob does not change the Rate underneath. An amount of zero or above leaves Rate alone. In the cord list, Lfo2Ck follows Lfo2Trg. The startup screen shows **LfoCk** and the time the image was built. This image is the released EOS 4.71-beta2 plus that cord.

Read this addendum together with the EOS 4.7 Operation Manual and the EOS 4.7 Operation Manual Addendum. Everything not mentioned here works as described there.

> **Note:** EOS 4.71-beta2 is not made, supported or endorsed by E-mu Systems or Creative Technology. It is a beta: back up your banks before installing it. You can return to EOS 4.70 at any time.

Names Changed in EOS 4.71-beta2

Feature	EOS 4.71-beta (beta1)	EOS 4.71-beta2
Transwave / wavetable scan cord destination	**SLpScn**	**WTScan**

Only the name has changed: the destination works the same way. Presets saved with an SLpScn cord in beta1 load and play unchanged in beta2 and show WTScan.

To Install EOS version 4.71-beta2

EOS 4.71-beta2 is built on your computer from E-mu's own EOS 4.70 update file (**EOS470.EXE**) and sent to the E4 over MIDI. No floppy disk is needed.

You need:

- E-mu's **EOS470.EXE** (the EOS 4.70 update file).
- A MIDI interface connected **both ways**: computer MIDI Out → E4 MIDI In, and E4 MIDI Out → computer MIDI In. The E4 answers every packet during the transfer.
- The **EOS Flasher** application (macOS, Windows or Linux).

First launch on a Mac. The apps are not signed by a registered Apple developer, so the first time you open one, macOS says **"EOS Flasher" Not Opened** ("Apple could not verify … is free of malware"). Click **Done**, not **Move to Trash**, then:

- **macOS 15 Sequoia and later:** open **System Settings** → **Privacy & Security**, scroll down to **Security** and click **Open Anyway** next to "EOS Flasher was blocked". Confirm with your password, then click **Open Anyway** again.
- **macOS 10.13 to 14:** Control-click (or right-click) the app, choose **Open**, then click **Open**.

This is needed once for each app (EOS Flasher and EOS Pan Test). On Windows, if SmartScreen appears, choose **More info** → **Run anyway**.

To build and send EOS 4.71-beta2 +LfoCk:

1. Start **EOS Flasher** and press **Choose EOS470.EXE…**. Select the file. The flasher confirms "original E-mu EOS v4.70 found and verified".
2. Select **EOS 4.71-beta2 +LfoCk** and press **Build**. (Optionally press **Save .syx…** to keep a copy.)
3. Select the MIDI **Output to E4** and the MIDI **Input from E4**.
4. Press **Send to E4**.
5. Turn the E4 on while holding the **Enter** button. When the display asks **"Download OS from MIDI?"**, press **Yes**. The display shows "Searching for OS…" and the transfer starts; the flasher shows its progress (about 9 minutes).
6. When the transfer is complete the E4 checks the new software and asks **"Burn to flash?"**. Press **Yes**. Do not turn the E4 off while it writes to flash.
7. The E4 restarts. The startup screen shows **EOS 4.71-beta2 +LfoCk**.

To return to EOS 4.70: repeat the procedure and select **Original EOS 4.70** in step 2. The flasher can also build the earlier **EOS 4.71-beta**.

> **Note:** If the flasher reports "header not acknowledged", check that the E4's MIDI Out is connected to the selected MIDI input and that the E4 is showing "Searching for OS…".

Filters: OB-Xa Models

The lowpass, highpass and bandpass filter types below now model the filters of the Oberheim OB-Xa, which are built on the Curtis CEM3320 filter chip. Filter type names and numbers are unchanged, so existing presets load as before and use the new models.

Filter type	4.71-beta2 model
2 Pole Lowpass	OB-Xa 2-pole state-variable lowpass (12 dB/oct)
4 Pole Lowpass	OB-Xa 4-pole ladder lowpass (24 dB/oct)
6 Pole Lowpass	4-pole ladder followed by the 2-pole, same cutoff (36 dB/oct)
2nd Order High-pass	OB-Xa 2-pole response as highpass
4th Order High-pass	OB-Xa 4-pole response as highpass
2nd Order Band-pass	OB-Xa state-variable bandpass

Filter type	4.71-beta2 model
4th Order Band-pass	OB-Xa 4-pole bandpass
Contrary Band-pass	E-mu's contrary bandpass with the cutoff calibrated to the display

- **Frequency:** the cutoff (or centre) frequency now follows the frequency shown on the display, also at high Q. In EOS 4.70 the lowpass peak drifted up to 25 % sharp at low frequencies and the highpass corner sat 25–60 % below the display.
- **Q:** resonance rises the way the OB-Xa's does. Maximum resonance is matched to EOS 4.70 (about 19 dB for 2-pole, 39 dB for 4-pole and 46 dB for 6-pole), so preset levels stay balanced.
- The 4th Order Band-pass has a resonant peak of about +15 dB at maximum Q.

> **Note:** The E4's filter hardware is linear, so the OB-Xa's saturation and self-oscillation are not reproduced.

Smoother resonance (new in 4.71-beta2): the OB-Xa lowpass, highpass and bandpass filters now change resonance smoothly through the whole **Q** range. In 4.71-beta, Q moved in steps of 4 or 8 values.

4 Pole LP+Fuzz (new in 4.71-beta2)

4 Pole LP+Fuzz is a new filter type: the OB-Xa 4-pole ladder lowpass with a drive stage that pushes the E4's filter chip into overload. It is the **last entry in the filter type list** (after Peak/Shelf Morph), so the numbers of all other filter types are unchanged.

- **Frequency** sets the lowpass cutoff, as on the 4 Pole Lowpass.
- **Q** (shown as **Fuzz**) sets the drive and the resonance together. At Q 0 the filter is clean. As Q rises, the resonance rises (half as much as on the 4 Pole Lowpass) and the drive increases until the filter clips.
- The fuzz is strongest at low and medium cutoffs. Around a cutoff of 96 it starts to clip near the cutoff frequency from about Q 64, and across the whole passband from about Q 96. At high cutoffs (200 and up) it stays clean.
- Frequency and Q can both be modulated in real time (FilFreq and FilRes cords).

LFO Tempo Sync

LFO 1 and LFO 2 can run in time with the master clock: the sequencer/arpeggiator tempo, or incoming MIDI clock.

To sync an LFO to tempo:

1. Edit the voice and go to the **LFO** page.
2. Turn the **Rate** field past its fastest free-running value (**18.01 Hz**). The display now shows a note value instead of hertz (see the table below).
3. Set **Sync** to **Key Sync** or **Free Run**.

Rate display	Cycle length	MIDI clocks per cycle
4bars	4 bars	384
2bars	2 bars	192
1bar	1 bar	96
1/2.	dotted half note	72
1/2	half note	48

Rate display	Cycle length	MIDI clocks per cycle
1/4.	dotted quarter note	36
1/2T	half-note triplet	32
1/4	quarter note	24
1/8.	dotted eighth note	18
1/4T	quarter-note triplet	16
1/8	eighth note	12
1/16.	dotted sixteenth note	9
1/8T	eighth-note triplet	8
1/16	sixteenth note	6
1/16T	sixteenth-note triplet	4
1/32	thirty-second note	3

Sync settings

- **Key Sync:** each note starts its LFO at the beginning of the cycle.
- **Free Run:** all voices run in phase with the song; the cycle is locked to bar 1 (MIDI Start).

Notes

- The tempo follows the master clock. To follow your sequencer, set the Master **Clock Source** to **External** and send MIDI clock to the E4.
- While an LFO is synced, the Rate cords (Lfo1Rt / Lfo2Rt), **Variation** and the LFO trigger are ignored. **Shape**, **Delay** and all LFO destinations work as usual.
- Over MIDI SysEx the divisions are Rate values 128–143.

New LFO shape: saw down (shape 16), a falling sawtooth. The existing **sawtooth** rises.

LFO Clock from a Cord (Lfo1Ck and Lfo2Ck)

Lfo1Ck and **Lfo2Ck** choose an LFO's tempo division from a cord. The divisions are the same sixteen lengths as a synced Rate, from 4 bars to 1/32. **Lfo1Ck** controls LFO 1 and **Lfo2Ck** controls LFO 2.

To control an LFO from a cord:

1. Edit the voice and go to the **Cords** page.
2. Choose a source, set the destination to **Lfo1Ck** or **Lfo2Ck**, and set the amount negative. An amount of **0** or above leaves that LFO's Rate in control, including a tempo-synced Rate.
3. Play a note and move the source.

At amount **−100**, a MIDI controller (the mod wheel, or a MIDI A–H source) selects the division below. Several cords to the same destination add. A smaller negative amount reaches only the slower divisions.

Controller value	Division
0	Rate unchanged
1–7	4bars
8–15	2bars

Controller value	Division
16–23	1bar
24–31	1/2.
32–39	1/2
40–47	1/4.
48–55	1/2T
56–63	1/4
64–71	1/8.
72–79	1/4T
80–87	1/8
88–95	1/16.
96–103	1/8T
104–111	1/16
112–119	1/16T
120–127	1/32

Notes

- While the cord is negative, that LFO restarts at the beginning of its cycle on each new note. The LFO **Sync** switch is not used. **Shape** and **Delay** still work.
- While the cord is negative, that LFO's **Rate** field shows the cord's division (the same names as a synced Rate, from 4bars to 1/32). Turning the knob does not change the stored Rate. At zero or above, Rate shows and edits as before. The field keeps its normal colour.
- While the cord is negative, the Rate cord (Lfo1Rt / Lfo2Rt), **Variation** and the LFO trigger are ignored, as they are when Rate itself is synced.
- The division follows the master clock, internal or external, the same way a synced Rate does.
- In the destination list, **Lfo1Ck** follows Lfo1Trg. **Lfo2Ck** follows Lfo2Trg, and **Lag0in** follows Lfo2Ck. A preset stores the destination number, which is unchanged.

MIDI Clock

Clock keeps running when the sequencer stops. With the Clock Source set to **External**, when your sequencer stops sending MIDI clock:

- synced LFOs, the clock modulation sources (**CkDwhl … Ck16th**), the arpeggiator and Beats keep running at the last tempo instead of freezing;
- the sequencer still stops with your DAW.

When the sequencer continues, synced LFOs return to its grid; MIDI **Start** resets them to bar 1. Short dropouts in the clock (under 250 ms) are caught up, so the LFOs stay in time.

Corrected in this version:

- With Clock Source External, EOS 4.70 counted the incoming MIDI clock **and** its own internal clock, so the sequencer, arpeggiator and clock sources ran too fast (60 BPM played like 180 BPM). They now follow the MIDI clock exactly.

- After power-on, the clock sources did not move until the arpeggiator or sequencer tempo had been changed. The clock now starts at the arpeggiator tempo when the first note plays.

Sample Edit: Insert Silence (new in 4.71-beta2)

Insert Silence adds 4 samples of silence at a point in a sample. Use it to give a loop or a wavetable the lead-in it needs: the E4 does not allow a loop to start before sample 2, so slices and transwave frames should start after a few samples of silence.

To insert silence:

1. In **Sample Edit**, press **Utils** (F1), then **SiInc** (F6).
2. The display shows "Paste Section" (Insert Silence works like Paste). Set **Side** to **Both**.
3. Press **OK** (F6).
4. Set the start point with the data wheel where the silence should go (usually the start of the sample).
5. Press **OK** (F6). Four samples of silence are inserted at that point.

Undo removes them again (it is labelled "Paste Section").

Transwave Scanning: the WTScan Cord Destination

> **Name change:** in EOS 4.71-beta (beta1) this destination is called **SLpScn** (sample loop scan). In EOS 4.71-beta2 it is called **WTScan** (wavetable scan). Only the name has changed: it works the same way, and presets made with SLpScn cords play unchanged and show WTScan.

The new cord destination **WTScan** (wavetable scan) slides a sample's loop window through the sample while the note plays. With a sample made of a series of single-cycle waveforms ("frames"), the loop moves from frame to frame: Ensoniq-style transwave and wavetable sounds.

To scan a wavetable sample:

1. Prepare the sample as described in **Building Transwave Samples** below: **at least 2 samples of silence**, then the frames. Loop it on its first frame (forward or alternating loop, Start 2).
2. Assign the sample to a voice.
3. On the **Cords** page, choose a source (for example **ModWl**, **Lfo1+** or an envelope), set the destination to **WTScan** and the amount to **+100**.
4. Play a note and move the source: the sound changes as the loop moves through the frames.

Notes:

- At **+100**, a full-scale source moves the loop from frame 1 to the end of the sample; **−100** moves it towards the start. Smaller amounts cover proportionally less; several cords add up.
- The loop position is updated every 10 ms and always stays inside the sample.
- Positions between frames blend the two neighbouring frames (no crossfade).
- The existing **SLoop** destination is unchanged (small loop offsets, about 127 samples per full cord).
- **New in 4.71-beta2:** a note now starts at the current loop position, not at the start of the sample.
- **New in 4.71-beta2:** fast source moves glide, at most 64 samples per 10 ms, so a fast envelope or a quick mod-wheel move does not make a burst of noise.

Building Transwave Samples

WTScan slides the sample's loop window through the sample while the note plays. How you build the sample decides how far it can travel and how many waveforms (frames) it can reach.

Rules

- **Minimum 2 samples of silence first.** Every transwave sample must begin with at least 2 samples of silence before frame 1: the E4 does not allow a loop to start before sample 2 (the loop editor stops at Start 2). Without this lead-in, frame 1 cannot be the loop, and the frame grid is off by 2 samples. With it, the loop sits on frame 1 at Start 2, End 2 + frame size.
- **Equal frames.** Make every frame exactly the loop (window) length. For click-free single-cycle frames, start each one at a zero crossing.
- **Loop on the first frame**, forward or alternating loop type. Other loop types are left alone.
- **Travel limit: 32,767 samples.** EOS keeps the loop shift in a 16-bit value, so the window can move at most 32,767 samples from where the loop is set (about 0.74 s at 44.1 kHz, 0.68 s at 48 kHz). The loop is also kept inside the sample.
- **Keep the whole sample within reach:** sample length ≤ 2 + 32,768 samples (with the loop on frame 1). Longer samples still play, but WTScan's range is scaled to the whole sample, so the part of the controller travel aimed beyond the reachable area just holds at the limit.

Maximum number of frames (loop on frame 1; frames × frame size ≤ 32,768)

Frame size (samples)	Max frames	Sample length	Frame-1 pitch at 44.1 kHz
64	512	32,770	689.1 Hz
128	256	32,770	344.5 Hz
256	128	32,770	172.3 Hz
512	64	32,770	86.1 Hz
1,024	32	32,770	43.1 Hz
2,048	16	32,770	21.5 Hz
4,096	8	32,770	10.8 Hz

Sample length = 2 + frames × frame size. Fewer frames are fine; the scan always spans the whole sample. Frame-1 pitch is just the loop's natural pitch (sample rate ÷ frame size); set the root key so it plays in tune. Bigger frames suit lower pitches and hold more detail. Smaller frames give more frames, but the E4's resampling dulls high harmonics more when a frame has few samples per cycle.

Control

- **Range:** one cord at **+100** from a full-scale source (mod wheel at 127, Lfo1+ at peak, an envelope at full) moves the window from frame 1 to the end of the sample; **−100** moves it towards the start. Smaller amounts cover proportionally less, and extra cords add together.
- **Resolution:** the window position updates every 10 ms. A MIDI controller has 128 steps, so on a 128 × 256 table each mod-wheel step moves about one frame (about 256 samples); on 32 × 256 it is about 62 samples per step.
- **Character:** positions between frames blend the two neighbours, with the slightly gritty sound of a moving loop. There is no crossfade.
- **Other loop cords:** stock SLoop cords on the same voice still add their small fine offset (about 127 samples per full cord).

Example: the test bank's **WT test** presets use a table of 32 frames of 256 samples with 2 lead-in samples (8,194 samples, loop 2–258). Use Mod Wheel → WTScan +100.

Beat Slicing: the 'Slice Cord Destination (new in 4.71-beta2)

The new cord destination **'Slice** plays one slice of a sample per key, like a sliced drum loop in a dedicated slicer. The slices are equal-length sections of one sample, so no new samples are made and no extra sample memory is used. As with **'SStart**, the apostrophe means the slice is chosen when the note starts.

To slice a loop:

1. Prepare the sample as equal-length slices, one after another, for example a one-bar loop at an exact tempo with 16 equal steps. Start it with **at least 2 samples of silence**: the E4 does not allow a loop to start before sample 2. **Insert Silence** (Sample Edit, Utils, Silnc) adds 4.
2. In Sample Edit, set the **loop** on the first slice: Loop Start = the start of slice 1, Loop End = the end of slice 1. The loop length is the slice length.
3. **Loop off:** each slice plays once and stops at its end.
4. **Loop on:** each slice loops on itself while the key is held.
5. Assign the sample to a voice. The voice's **Original Key** is the key that plays slice 1.
6. On the **Cords** page, set the source to **Key+**, the destination to **'Slice** and the amount to **+100**.
7. Play the Original Key for slice 1, the next key up for slice 2, and so on.

Notes:

- Keys above the last slice play the last slice; keys below the Original Key play the first slice (or slices before the loop, if the sample has whole slices before it).
- Stereo samples are sliced on both sides.
- **SStart** still works: it offsets the start point inside the slice.
- With a controller source, such as **ModWl** at +100, the controller value counted from the Original Key picks the slice: value = Original Key plays slice 1, each step up the next slice.
- The slice is set at note-on; moving the source during a note does not change the slice.
- 'Slice and **WTScan** share the same cord sum: do not use both on one voice.

'Slice and WTScan compared

Both destinations read the sample's loop, but they do different jobs: **'Slice** is a slicer, **WTScan** is a transwave scanner.

	'Slice	WTScan
When it acts	Once, at note-on	At note-on, then every 10 ms while the note plays
Where the note starts	At the start of slice n	At the current loop position (not the sample start)
What moves	Play start and end (and the loop, if on) jump to slice n	The loop window slides through the sample
Steps	Whole slices, snapped	Continuous: in-between positions blend neighbouring frames
Fast moves	—	Glide: at most 64 samples per 10 ms, so fast sources do not burst
Scale	One slice per key or controller step, from the Original Key	Full scale (+100, source at max) sweeps the whole sample
Loop off	Works: each slice plays once	Does nothing; it needs a forward loop

Use **'Slice** for sliced drum loops and phrases, and **WTScan** for evolving wavetable and transwave sounds. Do not use both on one voice: they share one cord sum, and on a voice with a 'Slice cord the WTScan scan is switched off.

Beat Munger: Slice to Samples and Preset (new in 4.71-beta2)

The new **Slice** soft key in Beat Munger turns a drum loop into a playable kit, like a dedicated loop slicer. It uses the slice points Beat Munger finds, makes one new sample per slice and builds a new preset with each slice on its own key.

To slice a loop:

1. In Sample Edit, select the loop and open **Beat Munger**.
2. Adjust Beat Munger's settings, using **re-Anlyz** and **Play**, until the slices fall on the hits you want.
3. Press **Slice** (the soft key between Cancel and Stop).

The E4 then:

- makes one new sample per slice in the first free sample locations, named after the loop with a slice number, for example `Amen Break 01`, `Amen Break 02`, ...;
- makes a new preset in the first free preset location, named after the loop;
- maps slice 1 to key 36 (C1), slice 2 to the next key up, and so on. Each slice's original key is its own key, so every slice plays back at its original pitch;
- leaves Beat Munger and selects the new preset, with the message **Sliced: n samples, preset p**.

Notes:

- The original loop is not changed, and the sample clipboard and the Sample Edit selection are kept.
- Each cut is placed just before its hit, so attacks are not clipped. It then moves to the nearest point where the waveform crosses zero, so slices start without a click.
- Slice 1 starts at the beginning of the loop, and the last slice runs to its end, so the slices play back to back as the complete loop.
- Slices are uneven in length because they follow the hits in the loop: a slice runs from one hit to the next.
- Hits closer than about 23 ms are kept in one slice. A preset holds at most 92 slices (keys 36 to 127).
- Mono and stereo loops can both be sliced.
- Each slice takes the loop's loop setting: if the loop is set to loop, so is every slice.
- Slicing uses new sample memory about the size of the loop.

Messages:

Message	Meaning
Slice: no slices to make	Beat Munger has found no slices, or the sample is too short.
Slice: no free sample slots/preset	There are not enough free sample locations for the slices, or no free preset. Nothing is changed.
Slice: not enough sample memory	There is not enough free sample memory. Nothing is made; Beat Munger closes.

If Beat Munger is still playing when you press Slice, slicing starts when playback stops.

Slicing compared

	Beat Munger Slice	'Slice cord
Slice points	At the hits Beat Munger finds	Equal lengths, set by the loop
What it makes	New samples and a new preset	Nothing: it plays sections of one sample
Sample memory	About one more copy of the loop	None
Editing slices	Each slice is its own sample	Edit the one sample

Use **Beat Munger Slice** for real drum loops with uneven hits, and the **'Slice** cord for loops cut into equal steps.

Pan

Corrected in this version: in EOS 4.70 every pan control reached hard left or hard right at half its range: voice **Pan** at −32/+32, Multimode (mixer) **Pan** at −32/+32, MIDI **controller 10** at 32/96, and **AmpPan** cords at about half their amount. The outer half of each control did nothing. Since 4.71-beta the full range is used:

Setting	−64 / CC 0	−32 / CC 32	−16 / CC 48	0 / CC 64	+16 / CC 80	+32 / CC 96	+63 / CC 127
Left − Right, EOS 4.70	+72 dB	+72 dB (hard)	+15 dB	−0.2 dB	−16 dB	−72 dB (hard)	−72 dB
Left − Right, EOS 4.71-beta	+72 dB	+16 dB	+6 dB	−0.2 dB	−7 dB	−16 dB	−72 dB
Left − Right, EOS 4.71-beta2	+72 dB	+16 dB	+6 dB	**0.0 dB**	−7 dB	−16 dB	−72 dB

Balanced centre (new in 4.71-beta2): in EOS 4.70 and 4.71-beta a centred pan came out about 0.24 dB louder on the right, because E-mu's pan table has no exact middle entry. In 4.71-beta2 the centre (pan 0 and −1, CC10 63 and 64) is exactly balanced; all other settings are unchanged.

Presets that relied on the old behaviour (pan set to ±32 for a hard-panned sound) now sit half-way between the centre and the side; set them to ±64 for a hard pan.

To test pan on your E4: load the test bank **Test Suite.E4B** (from the `testbank` folder of the project) and run the **EOS Pan Test** application (see Pan Test Application below).

Performance

The cord processing of every playing voice now stops after the last cord in use instead of always checking all 24 cords. The sound is identical. On a 64-voice test preset the time spent on voice control dropped by about 7 % (the cord stage by 29 %), leaving more processing time for the rest of the system.

Pan Test Application

EOS Pan Test (macOS, Windows and Linux) measures how far the E4's pan controls really pan.

To run the pan test:

1. Load the bank **Test Suite.E4B** into the E4 (from the project's `testbank` folder). The test uses preset **000** (a single sine voice on key 72).
2. Connect MIDI both ways, and connect the E4's main outputs (and, if you like, sub outputs) to an audio interface input.
3. Start **EOS Pan Test**. Select the MIDI output and input, the E4's MIDI device ID and channel, the audio device and the input pair(s) the E4 is connected to.
4. Choose the tests (voice Pan, Multimode Pan, MIDI CC10) and press **Run tests**.
5. The results table shows the left and right levels for every setting, and the summary tells you whether the pan bug is present ("hard left/right at half range") or fixed ("full pan range"), and whether the centre is balanced (4.71-beta2) or about 0.24 dB right-heavy (4.70 and 4.71-beta).
6. Press **Save CSV + WAV…** to keep the measurements, and **Report to GitHub…** to send them to the project as a report (your browser opens a pre-filled report for you to review and submit).

> **Note:** "Prepare preset" selects preset 000 for editing, turns its cords down and opens its filter so that only pan changes the level. These edits are not saved.

Known Limitations

- Synced LFOs do not follow MIDI Song Position Pointer; after Continue they resume from where the sequencer stopped.
- While the MIDI clock is stopped, the clock keeps the last tempo within about 0.2 % (LFOs) and 2 % (clock sources).
- Beat Munger **Slice** makes at most 92 slices (keys 36 to 127). A slice can start with a small click where the waveform has no zero crossing near the cut.
- **'Slice** and **WTScan** share one cord sum: do not use both on one voice.
- EOS 4.71-beta2 runs on Ultra models only, like EOS 4.70.